TRABAJO DE FIN DE MÁSTER

Máster en IA Aplicada a la Ciberseguridad

SIOPV

Sistema Inteligente de Orquestación y Priorización de Vulnerabilidades

Priorización Inteligente de Vulnerabilidades en Entornos CI/CD

Estudiante: **Carlos Val Souto**

Fecha de Entrega: **1 de marzo de 2026**

Versión del Documento: **2.0 (Revisada y Ampliada)**

Índice de Contenidos

Nota: Para visualizar el índice, haz clic derecho sobre este campo y selecciona "Actualizar campo" → "Actualizar toda la tabla".

Índice de Contenidos **2**
1. Resumen Ejecutivo **3**
1.1 Pilares de Innovación Técnica 3
2. Evolución Arquitectónica desde la Propuesta Original **4**
2.1 Tabla Comparativa de Evolución de Componentes 4
2.2 Justificación de la Migración CrewAI → LangGraph 4
3. Arquitectura Detallada del Pipeline (8 Fases) **6**
3.1 FASE 1: Ingesta y Preprocesamiento de Datos 6
Especificaciones Técnicas 6
Proceso Técnico Detallado 6
3.2 FASE 2: Enriquecimiento de Contexto (Dynamic RAG) 8
Especificaciones Técnicas 8
Integración con APIs Externas 8
Patrón CRAG (Corrective RAG) 8
Gestión de ChromaDB en Hardware Limitado 9
3.3 FASE 3: Clasificación y Score de Riesgo (Machine Learning) 10
Especificaciones Técnicas 10
Dataset de Entrenamiento 10
Vector de Features 10
Implementación XAI (Explicabilidad) 11
3.4 FASE 4: Orquestación y Gestión de Incertidumbre 12
Especificaciones Técnicas 12
Uncertainty Trigger (Umbral Adaptativo) 12
Checkpointing y Recuperación 12
3.5 FASE 5: Control de Acceso de Grano Fino (Zero Trust) 13
Especificaciones Técnicas 13
Modelo de Autorización ReBAC 13
Flujo de Verificación 13
3.6 FASE 6: Privacidad y Prevención de Fuga de Datos (DLP) 13
Especificaciones Técnicas 13
Arquitectura de Sanitización Dual 13
Integración con LangSmith (Logging Seguro) 14
3.7 FASE 7: Interfaz y Validación Humana (Human-in-the-Loop) 15
Especificaciones Técnicas 15
Tríada de Evidencia 15
Timeout Escalation 15
3.8 FASE 8: Acción y Trazabilidad (Output Layer) 15
Especificaciones Técnicas 15
Ticket Enriquecido en Jira 15
PDF de Auditoría 16
4. Resiliencia Operativa y Gestión de Fallos **17**

4.1 Circuit Breaker Pattern 17
4.2 Estrategias de Fallback por Componente 17
4.3 Rate Limit Management 17
5. Métricas de Evaluación y KPIs 19
5.1 Métricas del Modelo ML 19
5.2 Métricas Operativas del Sistema 19
5.3 Métricas de Explicabilidad (XAI) 19
6. Planificación Temporal (Timeline) 21
6.1 Cronograma Detallado 21
6.2 Gestión de Riesgos del Timeline 21
7. Matriz de Trazabilidad Técnica Final 22
8. Estándares de Desarrollo y Buenas Prácticas 23
8.1 Arquitectura 12-Factor App 23
8.2 Arquitectura Hexagonal (Ports & Adapters) 23
Estructura de Directorios 24
8.3 Principios SOLID 24
8.4 Calidad de Código Python 25
Type Hints y Validación Estática 25
Linting y Formatting 25
Testing Strategy 25
8.5 Gestión de Dependencias 25
8.6 Containerización y CI/CD 26
Docker Multi-Stage Build 26
Pipeline CI/CD (GitHub Actions) 26
8.7 Logging y Observabilidad 26
Structured Logging 26
Métricas y Tracing 27
8.8 Seguridad en el Desarrollo (DevSecOps) 27
OWASP Top 10 Mitigations 27
Gestión de Secrets 27
8.9 Git Workflow y Versionado 28
Conventional Commits 28
Branching Strategy (GitHub Flow) 28
8.10 Documentación 28
9. Stack Tecnológico Completo 29
9.1 Dependencias Python (pyproject.toml) 29
9.2 APIs Externas Requeridas 29
9.3 Consideraciones de Hardware (MacBook Air M3 16GB) 29
9.4 Despliegue en Cloud (Alternativa a Local) 29
Plataformas Recomendadas 29
Arquitectura Cloud Recomendada (Railway/Render) 30
Mapeo de Componentes Local → Cloud 30
10. Conclusiones y Siguientes Pasos 31
10.1 Siguientes Pasos Inmediatos 31

1. Resumen Ejecutivo

El proyecto SIOPV (Sistema Inteligente de Orquestación y Priorización de Vulnerabilidades) constituye una respuesta arquitectónica avanzada al problema de fatiga de alertas en los ciclos de desarrollo de software seguros (Secure SDLC). La premisa fundamental es que los escáneres de vulnerabilidades tradicionales, como Trivy, generan volúmenes de alertas que exceden la capacidad operativa de los equipos de seguridad, resultando en una priorización subóptima basada exclusivamente en métricas estáticas como CVSS.

SIOPV implementa una Capa de Inteligencia situada entre la Integración Continua (CI) y el Despliegue Continuo (CD), actuando como un filtro inteligente que transforma reportes masivos de CVEs en tickets de remediación priorizados y enriquecidos con contexto de explotabilidad real.

1.1 Pilares de Innovación Técnica

- **Hibridación ML + GenAI:** Combinación de algoritmos de Machine Learning clásico (XGBoost/Random Forest) para scoring determinista con Modelos de Lenguaje de Gran Escala (LLMs) para razonamiento semántico y contextualización dinámica.
- **IA Explicable (XAI):** Cada decisión de priorización está respaldada por justificación matemática mediante LIME (Local Interpretable Model-agnostic Explanations) y SHAP (SHapley Additive exPlanations), complementada con trazabilidad de razonamiento Chain-of-Thought.
- **Seguridad por Diseño:** Arquitectura Zero Trust con control de acceso de grano fino (ReBAC mediante OpenFGA) y prevención de fuga de datos (DLP mediante Microsoft Presidio + validación semántica).
- **Resiliencia Operativa:** Gestión de rate limits, circuit breakers, fallbacks y degradación graceful para garantizar disponibilidad en entornos de producción.

2. Evolución Arquitectónica desde la Propuesta Original

La propuesta inicial contemplaba un flujo lineal simplificado:

Trivy → JSON → Agentes (CrewAI) → RAG (Fuentes públicas) → Informe priorizado.

Tras un análisis de viabilidad técnica y requisitos de producción, se ha evolucionado hacia un Pipeline Híbrido GenAI + ML con las siguientes modificaciones fundamentales:

2.1 Tabla Comparativa de Evolución de Componentes

Componente Original	Evolución Técnica	Justificación
CrewAI	**LangGraph**	Control total sobre estados y ciclos deterministas. Permite Checkpointing granular y branching condicional basado en métricas de confianza (Uncertainty Trigger). Facilita depuración mediante exposición explícita del grafo de estados.
Agentes 1-4 (roles fijos)	**Nodos en Grafo + Multi-model**	Optimización de costes mediante selección dinámica de modelo según complejidad de tarea. Claude Haiku 4.5 para tareas estructurales, Claude Sonnet 4.5 para razonamiento crítico.
RAG Estático (corpus pre-indexado)	**Dynamic RAG (Just-in-Time)**	Recuperación bajo demanda de NVD/GitHub según el hallazgo específico del escáner. Elimina necesidad de mantener corpus masivo en memoria. Garantiza datos actualizados al segundo.
Informe Simple (texto plano)	**XAI Layer + DLP + Audit Trail**	Fundamentación matemática de decisiones (LIME/SHAP), protección contra fuga de datos sensibles (DLP), y trazabilidad completa para cumplimiento normativo (CoT Logging).
Sin gestión de fallos	**Circuit Breaker + Fallbacks**	Resiliencia ante fallos de APIs externas mediante patrón Circuit Breaker con backoff exponencial y estrategias de degradación graceful.

2 .2 Justificació n de la Migració n CrewAI → LangGraph

CrewAI implementa la orquestación de agentes mediante un paradigma de "conversaciones entre agentes", donde cada agente tiene un rol definido y se comunican mediante intercambio de mensajes. Este diseño, aunque intuitivo, presenta limitaciones críticas para un sistema de producción:

1. **No-determinismo inherente:** El orden de ejecución de agentes y el contenido de sus intercambios depende del LLM subyacente, dificultando la reproducibilidad de resultados.
2. **Checkpointing opaco:** La persistencia de estado intermedio requiere serializar conversaciones completas, no estados discretos del pipeline.
3. **Branching limitado:** Implementar lógica condicional (e.g., escalar a humano si confianza < 0.7) requiere hacks en el prompt, no control programático.
4. **Depuración compleja:** Trazar por qué un agente tomó una decisión específica implica analizar conversaciones multi-turno, no inspeccionar transiciones de estado.

LangGraph, en contraste, expone el grafo de estados como ciudadano de primera clase. Cada nodo representa una función pura que transforma el estado, los edges definen transiciones explícitas (condicionales o incondicionales), y el estado global es un objeto tipado (mediante TypedDict o Pydantic) que puede serializarse trivialmente para checkpointing.

3. Arquitectura Detallada del Pipeline (8 Fases)

El sistema SIOPV se estructura en 8 fases secuenciales con capacidad de branching condicional. Cada fase está implementada como un nodo en el grafo de LangGraph, con responsabilidades claramente delimitadas y contratos de entrada/salida definidos mediante schemas Pydantic.

3.1 FASE 1: Ingesta y Preprocesamiento de Datos

Especificaciones Técnicas

- **Módulo:** `Ingestion_Engine`
- **Modelo IA:** Claude Haiku 4.5 (claude-haiku-4-5-20251001)
- **Input:** Reporte JSON de Trivy (estructura Results[].Vulnerabilities[])
- **Output:** Lista de objetos VulnerabilityRecord validados y deduplicados

Proceso Técnico Detallado

Map-Reduce de JSON: El reporte de Trivy puede contener miles de vulnerabilidades distribuidas en múltiples targets (imágenes, filesystems). Para evitar saturación de la ventana de contexto del LLM, se implementa un patrón Map-Reduce:

- **Map:** Fragmentación del JSON en chunks de máximo 50 vulnerabilidades.
- **Process:** Cada chunk se procesa independientemente para extracción de campos críticos.
- **Reduce:** Consolidación de resultados en estructura unificada con deduplicación por CVE-ID.

Validación Pydantic: Cada vulnerabilidad extraída se valida contra un schema estricto que garantiza integridad de tipos y presencia de campos obligatorios:

- cve_id: str (formato CVE-YYYY-NNNNN)
- package_name: str
- installed_version: str
- fixed_version: Optional[str]
- severity: Literal["CRITICAL", "HIGH", "MEDIUM", "LOW", "UNKNOWN"]
- cvss_v3_score: Optional[float] (0.0-10.0)

Deduplicación Pre-Pipeline: Trivy puede reportar el mismo CVE múltiples veces (diferentes paths en el filesystem, diferentes layers de imagen Docker). Se implementa deduplicación basada en la tupla (cve_id, package_name, installed_version), conservando el primer registro encontrado y agregando metadata de ubicaciones adicionales.

Batch Processing por Paquete: Optimización de llamadas al LLM agrupando vulnerabilidades por package_name antes de invocar al modelo. Esto reduce el número de llamadas a la API y aprovecha mejor la ventana de contexto al proporcionar contexto de paquete coherente.

3.2 FASE 2: Enriquecimiento de Contexto (Dynamic RAG)

Especificaciones Técnicas

- **Módulo:** `Dynamic_RAG_Researcher`
- **Modelo IA:** Claude Sonnet 4.5 (claude-sonnet-4-5-20250929)
- **Fuentes de Datos:** NVD API, GitHub Security Advisories API, FIRST EPSS API, Tavily Search API
- **Almacenamiento:** ChromaDB (modo híbrido: SQLite para persistencia + cache en memoria)

Integración con APIs Externas

NVD (National Vulnerability Database): Fuente primaria de información estructurada sobre CVEs. Se consulta el endpoint /cves/2.0 con el parámetro cveId para obtener descripción completa, vectores CVSS, referencias y CPE (Common Platform Enumeration) afectados.

  - **Rate Limit sin API Key:** 5 requests/30 segundos
  - **Rate Limit con API Key:** 50 requests/30 segundos (recomendado obtener key gratuita)

GitHub Security Advisories: Información de vulnerabilidades específica del ecosistema de paquetes open source. Proporciona datos sobre versiones afectadas, versiones parcheadas y workarounds temporales no disponibles en NVD.

  - **Rate Limit sin auth:** 60 requests/hora
  - **Rate Limit con token:** 5,000 requests/hora

FIRST EPSS (Exploit Prediction Scoring System): Score de probabilidad de explotación activa en los próximos 30 días. Esencial para priorización basada en riesgo real, no teórico. Se actualiza diariamente.

  - **Endpoint:** https://api.first.org/data/v1/epss?cve={CVE-ID}
  - **Campos relevantes:** epss (probabilidad 0-1), percentile (ranking relativo)

Tavily Search API: Motor de búsqueda optimizado para agentes de IA. Se activa como fallback cuando las fuentes estructuradas (NVD, GitHub) no proporcionan información suficiente sobre exploits activos o PoCs públicos.

Patrón CRAG (Corrective RAG)

Para garantizar la calidad de los documentos inyectados en el contexto del LLM, se implementa el patrón CRAG con un nodo evaluador que califica la relevancia y veracidad de cada documento recuperado:

5. **Recuperación inicial:** Query paralela a NVD + GitHub + EPSS para cada CVE.
6. **Evaluación de relevancia:** Claude Sonnet evalúa si los documentos recuperados son relevantes para la vulnerabilidad específica (score 0-1).
7. **Decisión condicional:** Si relevancia < 0.6, se activa búsqueda OSINT complementaria via Tavily.
8. **Consolidación:** Documentos validados se almacenan en ChromaDB con embeddings para recuperación futura.

Gestión de ChromaDB en Hardware Limitado

Dado el hardware objetivo (MacBook Air M3 16GB), se implementa una estrategia híbrida de almacenamiento:

- **Persistencia en SQLite:** Los embeddings se persisten en disco para evitar re-cómputo entre ejecuciones.
- **Cache LRU en memoria:** Las 1000 consultas más frecuentes se mantienen en RAM para latencia mínima.
- **Eviction policy:** Si memoria > 4GB, se descargan embeddings menos usados a disco.

3.3 FASE 3: Clasificación y Score de Riesgo (Machine Learning)

Especificaciones Técnicas

- **Módulo:** `ML_Risk_Classifier`
- **Algoritmo Principal:** XGBoost (Gradient Boosting) con hiperparámetros optimizados via Optuna
- **Algoritmo Baseline:** Random Forest para comparación y ensemble opcional
- **Framework XAI:** SHAP (análisis global) + LIME (explicaciones locales)

Dataset de Entrenamiento

El modelo ML requiere un dataset etiquetado con vulnerabilidades clasificadas como "explotadas" vs "no explotadas". Se propone la siguiente estrategia de construcción:

Fuente Primaria - CISA KEV (Known Exploited Vulnerabilities): Catálogo oficial del gobierno de EE.UU. con CVEs confirmados como explotados activamente. Proporciona ground truth de alta confianza para la clase positiva.

  - **URL:** https://www.cisa.gov/known-exploited-vulnerabilities-catalog
  - **Tamaño aproximado:** ~1,200 CVEs (enero 2026)

Fuente Secundaria - EPSS Historical Data: Correlación de scores EPSS históricos con explotación confirmada para enriquecer el dataset con CVEs de alta probabilidad no incluidos en KEV.

Clase Negativa - Muestreo Estratificado de NVD: Selección de CVEs no presentes en KEV con EPSS < 0.1 y antigüedad > 2 años sin reportes de explotación. Ratio objetivo: 1:3 (positivos:negativos) para manejar desbalance.

Estrategia de Balanceo: Dado el desbalance inherente (pocas CVEs explotadas vs miles no explotadas), se implementarán técnicas de balanceo:

  - **SMOTE (Synthetic Minority Over-sampling Technique):** Generación de muestras sintéticas de la clase minoritaria.
  - **Class Weighting:** Asignación de pesos inversamente proporcionales a la frecuencia de clase en la función de pérdida.

Vector de Features

El modelo ML recibe un vector numérico con las siguientes características para cada vulnerabilidad:

Feature	Tipo	Descripción
cvss_base_score	float [0-10]	Score CVSS v3.1 base
attack_vector	int [0-3]	Vector de ataque: Network(3), Adjacent(2), Local(1), Physical(0)
attack_complexity	int [0-1]	Complejidad: Low(1), High(0)
privileges_required	int [0-2]	Privilegios: None(2), Low(1), High(0)

user_interaction	int [0-1]	Interacción usuario: None(1), Required(0)
scope	int [0-1]	Cambio de scope: Changed(1), Unchanged(0)
confidentiality_impact	int [0-2]	Impacto confidencialidad: High(2), Low(1), None(0)
integrity_impact	int [0-2]	Impacto integridad: High(2), Low(1), None(0)
availability_impact	int [0-2]	Impacto disponibilidad: High(2), Low(1), None(0)
epss_score	float [0-1]	Probabilidad de explotación en 30 días (FIRST EPSS)
epss_percentile	float [0-1]	Ranking percentil del EPSS score
days_since_publication	int	Días desde publicación del CVE
has_exploit_ref	int [0-1]	Existencia de referencias a exploits en NVD
cwe_category	int (encoded)	Categoría CWE codificada (target encoding)

Implementación XAI (Explicabilidad)

SHAP (Análisis Global): Generación de gráficos de importancia de features para justificar la política de seguridad general del modelo. Permite responder preguntas como: "¿Por qué el modelo prioriza vulnerabilidades con Network Attack Vector?"

LIME (Explicaciones Locales): Generación de explicaciones individuales por cada vulnerabilidad. Si el modelo marca un CVE como "Alto Riesgo", LIME extrae qué feature específico disparó la puntuación (e.g., "EPSS > 0.7 contribuyó +0.35 al score").

La salida de esta fase es una tupla (risk_probability, shap_values, lime_explanation) que se propaga al estado global de LangGraph para uso en fases posteriores.

3.4 FASE 4: Orquestación y Gestión de Incertidumbre

Especificaciones Técnicas

- **Módulo:** `LangGraph_Orchestrator`
- **Modelo IA:** Claude Sonnet 4.5 (orquestación de estado)
- **Persistencia:** SQLite para checkpointing de estado del grafo

Uncertainty Trigger (Umbral Adaptativo)

El Uncertainty Trigger compara la predicción numérica del modelo ML con el análisis semántico del LLM. La versión mejorada implementa un umbral adaptativo en lugar de un valor fijo:

9. **Cálculo de discrepancia:** discrepancy = |ml_score - llm_confidence|
10. **Umbral dinámico:** threshold = percentile_90(historical_discrepancies)
11. **Decisión:** if discrepancy > threshold OR llm_confidence < 0.7 → escalate_to_human
12. **Actualización:** El umbral se recalcula semanalmente basado en las últimas 500 evaluaciones.

Este enfoque adaptativo evita los problemas del umbral fijo (30%) que podría generar falsos positivos/negativos sistemáticos.

Checkpointing y Recuperación

El estado completo del grafo se persiste en SQLite tras cada transición de nodo, permitiendo:

- **Reanudación tras fallos:** Si la API de Claude falla mid-pipeline, el sistema puede reanudar desde el último checkpoint.
- **Intervención humana asíncrona:** El analista puede revisar casos escalados horas después sin perder contexto.
- **Auditoría post-mortem:** Reconstrucción completa del flujo de decisiones para análisis forense.

3.5 FASE 5: Control de Acceso de Grano Fino (Zero Trust)

Especificaciones Técnicas

- **Módulo:** `Authorization_Gatekeeper`
- **Tecnología:** OpenFGA (Fine-Grained Authorization)
- **Metodología:** ReBAC (Relationship-Based Access Control)

Modelo de Autorización ReBAC

A diferencia de RBAC (Role-Based Access Control), ReBAC define permisos basados en relaciones entre entidades. Esto permite expresar políticas como:

- *"El usuario X puede ver vulnerabilidades del proyecto Y si es propietario del proyecto"*
- *"El usuario X puede remediar vulnerabilidades si es analista asignado al activo afectado"*
- *"El usuario X puede exportar reportes si tiene relación de auditor con la organización"*

Flujo de Verificación

13. El nodo Authorization_Gatekeeper intercepta cada request antes de ejecutar operaciones sensibles.
14. Se construye una query de autorización: check(user:X, relation:viewer, object:project:Y)
15. OpenFGA evalúa la query contra el grafo de relaciones almacenado.
16. Si allowed=true, la operación procede. Si allowed=false, se retorna 403 Forbidden con audit log.

3.6 FASE 6: Privacidad y Prevención de Fuga de Datos (DLP)

Especificaciones Técnicas

- **Módulo:** `Data_Privacy_Guardrail`
- **Capa 1 (Determinista):** Microsoft Presidio (Regex + NLP)
- **Capa 2 (Semántica):** Claude Haiku 4.5 (validación contextual)

Arquitectura de Sanitización Dual

Capa 1 - Microsoft Presidio (Detección Determinista): Detecta y anonimiza patrones conocidos mediante Regex y modelos NLP pre-entrenados:

  - Direcciones IP (IPv4/IPv6)
  - Direcciones email
  - Tokens de API y credenciales (patrones comunes)
  - PII: números de teléfono, SSN, tarjetas de crédito
  - URLs internas y paths de filesystem

Capa 2 - Claude Haiku (Detección Semántica): Revisa el texto ya sanitizado por Presidio para detectar información sensible que no sigue patrones fijos:

- Nombres de proyectos internos
- Nombres de clientes o partners
- Secretos comerciales mencionados en contexto
- Información de arquitectura interna que no debe exponerse

Integración con LangSmith (Logging Seguro)

Para evitar conflictos entre la capa de DLP y el logging en LangSmith, se implementa el siguiente flujo:

17. Todo output del pipeline pasa por Data_Privacy_Guardrail ANTES de cualquier operación de logging.
18. Los logs enviados a LangSmith contienen únicamente datos sanitizados.
19. Opcionalmente, se puede usar logging local (ficheros) para trazas completas no sanitizadas, con acceso restringido.

3.7 FASE 7: Interfaz y Validación Humana (Human-in-the-Loop)

Especificaciones Técnicas

- **Módulo:** `Streamlit_Dashboard`
- **Framework:** Streamlit (Python)
- **Integración:** Polling a SQLite para detectar casos escalados

Tríada de Evidencia

Cuando un caso es escalado al dashboard, el analista visualiza tres componentes de evidencia:

20. **Resumen de IA:** Síntesis generada por Claude Sonnet con el contexto enriquecido de NVD, GitHub y OSINT.
21. **Gráfico LIME:** Visualización de barras mostrando qué features contribuyeron al score del modelo ML.
22. **Chain-of-Thought Log:** Trazabilidad completa del razonamiento del LLM, incluyendo documentos consultados y conclusiones intermedias.

Timeout Escalation

Para evitar que casos escalados queden indefinidamente sin resolución, se implementa lógica de timeout:

- **Timeout Nivel 1 (4 horas):** Notificación al analista asignado via email/Slack.
- **Timeout Nivel 2 (8 horas):** Escalado automático al Security Lead del equipo.
- **Timeout Nivel 3 (24 horas):** Auto-aprobación con flag de "revisión pendiente" para auditoría posterior.

3.8 FASE 8: Acción y Trazabilidad (Output Layer)

Especificaciones Técnicas

- **Módulo:** `Audit_Action_Engine`
- **Integración Ticketing:** Jira REST API v3
- **Generación PDF:** FPDF2 (Python)

Ticket Enriquecido en Jira

Cada vulnerabilidad procesada genera un ticket en Jira con la siguiente estructura:

- **Summary:** [CVE-XXXX-XXXXX] {Package} - {Severity} Risk
- **Description:** Resumen técnico sanitizado por DLP + recomendación de remediación.
- **Priority:** Mapeada desde el risk_score del modelo ML.
- **Labels:** security-vulnerability, auto-triaged, [affected-component]
- **Custom Fields:** CVSS Score, EPSS Score, SIOPV Confidence, Remediation Deadline.

PDF de Auditoría

Para cumplimiento normativo (ISO 27001, SOC 2, etc.), se genera un PDF de auditoría con:

- Listado completo de vulnerabilidades procesadas con timestamps.
- Decisiones de priorización con justificación (LIME summary).

- Casos escalados a humano con resolución y tiempo de respuesta.
- Chain-of-Thought completo (opcional, según requisitos de auditor).

4. Resiliencia Operativa y Gestión de Fallos

Un sistema de producción requiere estrategias de resiliencia ante fallos de dependencias externas. SIOPV implementa los siguientes mecanismos:

4.1 Circuit Breaker Pattern

Para cada API externa (Claude, NVD, GitHub, EPSS, Tavily), se implementa un circuit breaker con tres estados:

- **CLOSED:** Operación normal, requests fluyen hacia la API.
- **OPEN:** Tras N fallos consecutivos (default: 5), el circuit se abre y retorna fallback inmediatamente sin intentar la API.
- **HALF-OPEN:** Tras timeout (default: 60s), se permite un request de prueba. Si éxito → CLOSED, si fallo → OPEN.

4.2 Estrategias de Fallback por Componente

Componente	Escenario de Fallo	Estrategia de Fallback
Claude API	Rate limit, timeout, 5xx	Retry con backoff exponencial (máx 3). Si persiste, marcar vulnerabilidad para revisión manual.
NVD API	Rate limit excedido	Usar cache local de últimas 24h. Si cache miss, encolar para retry posterior.
GitHub API	Auth token inválido	Degradar a modo sin auth (60 req/h). Alertar a admin para refresh de token.
EPSS API	API no disponible	Usar último score conocido con flag de "stale data". Continuar pipeline.
Tavily API	Quota excedida	Omitir búsqueda OSINT. Proceder con datos de NVD/GitHub únicamente.
ML Model	Error en inferencia	Usar heurística basada en CVSS + EPSS. Marcar como "degraded confidence".
ChromaDB	OOM / Disk full	Evict LRU cache. Si persiste, operar en modo "no-cache" con mayor latencia.

4.3 Rate Limit Management

Se implementa un Rate Limiter centralizado que coordina las llamadas a todas las APIs externas:

- **Token Bucket Algorithm:** Cada API tiene un bucket con capacidad igual a su rate limit y tasa de recarga proporcional.
- **Request Queuing:** Si bucket vacío, requests se encolan en lugar de fallar inmediatamente.
- **Priority Scheduling:** CVEs con CVSS >= 9.0 tienen prioridad en la cola de requests.

5. Métricas de Evaluación y KPIs

Para demostrar el valor del sistema SIOPV, se definen los siguientes KPIs cuantificables:

5.1 Métricas del Modelo ML

Métrica	Objetivo	Justificación
Precision (clase positiva)	>= 0.85	Minimizar falsos positivos para evitar fatiga de alertas.
Recall (clase positiva)	>= 0.90	Crítico no perder vulnerabilidades realmente explotables.
F1-Score	>= 0.87	Balance entre precision y recall.
AUC-ROC	>= 0.92	Capacidad de discriminación del modelo.
Calibration Error	<= 0.05	Probabilidades predichas deben reflejar frecuencias reales.

5.2 Métricas Operativas del Sistema

Métrica	Objetivo	Justificación
Tiempo medio de triaje	< 5 minutos/CVE	Reducción significativa vs triaje manual (~30 min/CVE).
Reducción de falsos positivos	>= 60%	vs baseline de priorización solo por CVSS.
Tasa de escalado a humano	< 15%	El sistema debe resolver automáticamente la mayoría de casos.
Tiempo de respuesta p95	< 30 segundos	Latencia end-to-end del pipeline completo.
Disponibilidad del sistema	>= 99.5%	Uptime mensual considerando degradación graceful.

5.3 Métricas de Explicabilidad (XAI)

Métrica	Objetivo	Justificación
Fidelity Score (LIME)	>= 0.80	La explicación local debe aproximar bien al modelo original.
Consistency Score (SHAP)	>= 0.90	Explicaciones consistentes para inputs similares.
User Trust Rating	>= 4.0/5.0	Encuesta a analistas sobre confianza en explicaciones.

6. Planificación Temporal (Timeline)

Considerando la fecha de entrega del 1 de marzo de 2026 y la fecha actual (15 de enero de 2026), se dispone de aproximadamente 6.5 semanas de desarrollo efectivo. Se propone la siguiente distribución:

6.1 Cronograma Detallado

Semana	Fechas	Entregables
Semana 1	15-21 enero	Fase 1 (Ingesta): Parser de Trivy, validación Pydantic, deduplicación. Tests unitarios.
Semana 2	22-28 enero	Fase 2 (RAG): Integración NVD/GitHub/EPSS APIs, ChromaDB setup, patrón CRAG.
Semana 3	29 ene - 4 feb	Fase 3 (ML): Dataset construction, entrenamiento XGBoost, integración LIME/SHAP.
Semana 4	5-11 febrero	Fase 4 (Orquestación): Grafo LangGraph completo, Uncertainty Trigger, checkpointing SQLite.
Semana 5	12-18 febrero	Fases 5-6 (Seguridad): OpenFGA setup, Presidio integration, capa DLP semántica.
Semana 6	19-25 febrero	Fases 7-8 (UI/Output): Dashboard Streamlit, integración Jira, generación PDF auditoría.
Buffer	26 feb - 1 mar	Testing end-to-end, corrección de bugs, documentación final, preparación defensa.

6.2 Gestión de Riesgos del Timeline

Riesgo	Probabilidad	Mitigación
Retraso en integración de APIs	Media	Usar mocks durante desarrollo. Integrar APIs en paralelo, no secuencialmente.
Dataset ML insuficiente	Baja	CISA KEV está bien documentado. Alternativa: usar modelo pre-entrenado con fine-tuning.
Complejidad de LangGraph	Media	Empezar con grafo simplificado (4 nodos). Añadir complejidad incrementalmente.
Limitaciones de hardware	Baja	ChromaDB híbrido ya diseñado. Opción de usar embeddings en API si necesario.

7. Matriz de Trazabilidad Técnica Final

La siguiente tabla resume los componentes tecnológicos de cada fase del pipeline, con los modelos y métricas de calidad asociados:

Fase	Tecnología Clave	Modelo/Algoritmo	XAI / Métrica
1. Ingesta	Python / Pydantic	Claude Haiku 4.5	Schema Validation
2. Búsqueda	ChromaDB / Tavily	Claude Sonnet 4.5	CRAG Relevance
3. Priorización	Scikit-Learn / XGBoost	XGBoost + Optuna	LIME / SHAP
4. Orquestación	LangGraph / SQLite	Claude Sonnet 4.5	Uncertainty Score
5. Autorización	OpenFGA	ReBAC Model	Fine-Grain AC
6. Privacidad	Presidio / Haiku	Claude Haiku 4.5	DLP Guard
7. Decisión	Streamlit	Human-in-Loop	Trust Rating
8. Auditoría	Jira API / FPDF2	LangSmith	CoT Logging

8. Estándares de Desarrollo y Buenas Prácticas

El proyecto SIOPV se desarrolla siguiendo los estándares más actuales de la industria del software, garantizando mantenibilidad, escalabilidad, seguridad y portabilidad. Esta sección documenta los principios arquitectónicos, patrones de diseño y herramientas de calidad que rigen todo el desarrollo.

8.1 Arquitectura 12-Factor App

SIOPV implementa los 12 factores definidos por Heroku como estándar para aplicaciones cloud-native, permitiendo que el mismo código se ejecute en entornos locales, staging y producción sin modificaciones:

Factor	Principio	Implementación en SIOPV
I. Codebase	Un repositorio, múltiples deploys	Monorepo Git con CI/CD para dev/staging/prod
II. Dependencies	Declaración explícita de dependencias	Poetry/uv con pyproject.toml y lock file
III. Config	Configuración en variables de entorno	Pydantic Settings + .env (nunca hardcodeado)
IV. Backing Services	Servicios como recursos conectables	PostgreSQL, Redis, ChromaDB via URLs
V. Build/Release/Run	Separación estricta de etapas	Docker multi-stage + GitHub Actions
VI. Processes	Procesos stateless	Estado en PostgreSQL/Redis, no en memoria
VII. Port Binding	Exportar servicios via puerto	FastAPI en $PORT, Streamlit en $DASHBOARD_PORT
VIII. Concurrency	Escalar mediante procesos	Uvicorn workers, async/await para I/O
IX. Disposability	Inicio rápido, shutdown graceful	Lifespan events en FastAPI, signal handlers
X. Dev/Prod Parity	Entornos lo más similares posible	Docker Compose local replica servicios cloud
XI. Logs	Logs como streams de eventos	Structlog a stdout, agregación externa
XII. Admin Processes	Tareas admin como procesos one-off	CLI con Typer para migraciones y scripts

8.2 Arquitectura Hexagonal (Ports & Adapters)

El diseño interno de SIOPV sigue el patrón de Arquitectura Hexagonal, separando la lógica de negocio de los detalles de infraestructura. Esto permite sustituir componentes externos (bases de datos, APIs, interfaces) sin modificar el núcleo del sistema:

- **Domain Layer (Centro):** Entidades de negocio (Vulnerability, RiskScore, AuditLog), reglas de dominio puras sin dependencias externas.
- **Application Layer (Casos de Uso):** Orquestación de flujos (PrioritizeVulnerabilityUseCase, EnrichContextUseCase). Depende solo de interfaces (ports).

- **Ports (Interfaces):** Contratos abstractos: VulnerabilityRepository, LLMClient, VectorStore, NotificationService.
- **Adapters (Implementaciones):** PostgresVulnerabilityRepository, AnthropicLLMClient, ChromaVectorStore, JiraNotificationService.

Estructura de Directorios

```
siopv/├── domain/                    # Capa de dominio (sin dependencias)│   ├── entities/
# Vulnerability, RiskAssessment, etc.│   ├── value_objects/         # CVEId, EPSSScore, CVSSVector│
├── services/              # Reglas de negocio puras│   └── exceptions.py          # Excepciones de
dominio├── application/               # Capa de aplicación│   ├── ports/                 # Interfaces
abstractas (ABC)│   │   ├── repositories.py    # VulnerabilityRepository, etc.│   │   ├──
llm_client.py      # LLMClient interface│   │   └── vector_store.py    # VectorStore interface│   ├──
use_cases/             # Casos de uso│   │   ├── ingest_trivy.py│   │   ├── enrich_context.py│   │
├── classify_risk.py│   │   └── generate_report.py│   └── services/              # Servicios de
aplicación├── adapters/                  # Implementaciones concretas│   ├── persistence/           #
PostgreSQL, SQLite adapters│   ├── llm/                   # Anthropic, OpenAI adapters│   ├──
vectorstore/           # ChromaDB, Pinecone adapters│   ├── external_apis/         # NVD, GitHub, EPSS
clients│   └── notification/          # Jira, Slack adapters├── infrastructure/            #
Configuración y bootstrap│   ├── config/                # Settings, environment│   ├── di/
# Dependency injection container│   ├── logging/               # Structured logging setup│   └──
middleware/            # Auth, error handling├── interfaces/                # Puntos de entrada│   ├──
api/                   # FastAPI routes│   ├── cli/                   # Typer commands│   └──
dashboard/             # Streamlit app└── tests/                     # Tests por capa    ├── unit/
├── integration/    └── e2e/
```

8.3 Principios SOLID

Todo el código Python sigue los cinco principios SOLID para garantizar mantenibilidad y extensibilidad:

Principio	Definición	Aplicación en SIOPV
S	Single Responsibility	Cada módulo tiene una única razón de cambio. TriviParser solo parsea, no valida ni enriquece.
O	Open/Closed	Nuevos LLM providers se añaden creando adapters, sin modificar use cases existentes.
L	Liskov Substitution	PostgresRepo y SQLiteRepo son intercambiables donde se espera VulnerabilityRepository.
I	Interface Segregation	Interfaces pequeñas y específicas: LLMClient no incluye métodos de embedding.
D	Dependency Inversion	Use cases dependen de abstracciones (ports), no de implementaciones concretas.

8.4 Calidad de Código Python

Se implementa un pipeline de calidad de código automatizado mediante pre-commit hooks y CI/CD:

Type Hints y Validación Estática

- **Type Hints (PEP 484/604):** 100% del código tipado. Uso de tipos modernos: str | None en lugar de Optional[str].
- **Mypy (strict mode):** Verificación estática de tipos en CI. Configuración: --strict --ignore-missing-imports.
- **Pydantic v2:** Validación en runtime de todos los modelos de datos, DTOs y configuración.

Linting y Formatting

- **Ruff:** Linter ultrarrápido que reemplaza Flake8, isort, pyupgrade, y otros. Reglas habilitadas: E, F, W, I, N, UP, B, A, C4, DTZ, T10, EM, ISC, ICN, PIE, PT, Q, RSE, RET, SIM, TID, ARG, ERA, PL, TRY, RUF.
- **Black:** Formatter determinista. Line length: 100. Target version: py312.
- **Pre-commit:** Hooks ejecutados antes de cada commit: ruff, black, mypy, detect-secrets, trailing-whitespace.

Testing Strategy

- **Framework:** pytest con plugins: pytest-asyncio, pytest-cov, pytest-mock, pytest-xdist.
- **Coverage mínimo:** 80% global, 90% en domain layer. Reportes en Codecov.
- **Unit Tests:** Testean funciones puras y lógica de dominio en aislamiento. Mocks para dependencias externas.
- **Integration Tests:** Testean interacción entre capas con bases de datos en memoria (SQLite) y mocks de APIs.
- **E2E Tests:** Pipeline completo con Testcontainers (PostgreSQL, ChromaDB reales en Docker).

8.5 Gestión de Dependencias

Se utiliza uv (o Poetry como alternativa) para gestión moderna de dependencias Python:

- **pyproject.toml:** Fichero único de configuración siguiendo PEP 517/518/621. Define metadata, dependencias, y configuración de herramientas.
- **Lock file:** uv.lock (o poetry.lock) versionado en Git para builds reproducibles.
- **Grupos de dependencias:** Separación entre [dependencies], [dev-dependencies], [test-dependencies].
- **Renovate/Dependabot:** Actualización automática de dependencias con PRs semanales.

8.6 Containerización y CI/CD

Docker Multi-Stage Build

Imagen Docker optimizada siguiendo mejores prácticas de seguridad y tamaño:

- **Base image:** python:3.12-slim (Debian-based, sin vulnerabilidades conocidas).
- **Multi-stage:** Stage 1 (builder) instala dependencias, Stage 2 (runtime) copia solo lo necesario.
- **Non-root user:** Ejecución como usuario 'appuser' (UID 1000), nunca como root.
- **Health checks:** HEALTHCHECK definido para orquestadores (Kubernetes, Docker Swarm).

Pipeline CI/CD (GitHub Actions)

Stage	Trigger	Acciones
Lint	Push a cualquier rama	Ruff check, Black check, Mypy strict
Test	Push a cualquier rama	pytest con coverage, upload a Codecov
Security	Push + PR a main	Trivy scan (vulnerabilidades), Bandit (SAST), detect-secrets
Build	PR a main	Docker build, push a GitHub Container Registry (ghcr.io)
Deploy Staging	Merge a main	Deploy automático a Railway/Render staging environment

Deploy Prod	Release tag (vX.Y.Z)	Deploy a producción con approval manual

8.7 Logging y Observabilidad

Structured Logging

Se implementa logging estructurado en formato JSON para facilitar agregación y análisis:

- **Librería:** structlog con procesadores para contexto, timestamps ISO 8601, y formateo JSON.
- **Correlation IDs:** Cada request recibe un UUID propagado a través de todo el pipeline para trazabilidad.
- **Sensitive data masking:** Procesador personalizado que redacta API keys, tokens y PII antes de emitir logs.

Métricas y Tracing

- **OpenTelemetry:** Instrumentación automática de FastAPI, httpx, SQLAlchemy para tracing distribuido.
- **LangSmith:** Tracing específico de LLM calls: tokens, latencia, Chain-of-Thought, costes.
- **Health endpoints:** /health (liveness), /ready (readiness) con checks de dependencias.

8.8 Seguridad en el Desarrollo (DevSecOps)

OWASP Top 10 Mitigations

Vulnerabilidad OWASP	Mitigación en SIOPV
A01: Broken Access Control	OpenFGA (ReBAC) para autorización de grano fino en cada endpoint.
A02: Cryptographic Failures	TLS 1.3 obligatorio. Secrets en variables de entorno, nunca en código.
A03: Injection	SQLAlchemy ORM (no raw SQL). Pydantic valida todos los inputs.
A04: Insecure Design	Arquitectura Hexagonal con threat modeling en fase de diseño.
A05: Security Misconfiguration	Docker rootless, headers de seguridad (HSTS, CSP), Trivy scan en CI.
A06: Vulnerable Components	Dependabot/Renovate para updates. pip-audit en CI para CVEs conocidos.
A07: Auth Failures	JWT con rotación de keys. Rate limiting en endpoints de auth.
A08: Data Integrity Failures	Checksums en artefactos. Firma de commits (GPG). SLSA Level 2.
A09: Logging Failures	Structured logging con audit trail. Logs inmutables en producción.
A10: SSRF	Allowlist de dominios para APIs externas. No user-controlled URLs.

Gestión de Secrets

- **Local:** .env (ignorado en .gitignore). Template en .env.example sin valores reales.
- **CI/CD:** GitHub Secrets (encriptados, no visibles en logs).
- **Cloud:** Variables de entorno del proveedor (Railway Secrets, Render Environment).

- **Detección:** detect-secrets en pre-commit para prevenir commits accidentales de credenciales.

8.9 Git Workflow y Versionado

Conventional Commits

Todos los commits siguen la especificación Conventional Commits para generación automática de changelogs y versionado semántico:

- `feat:` Nueva funcionalidad (incrementa MINOR version)
- `fix:` Corrección de bug (incrementa PATCH version)
- `feat!: / BREAKING CHANGE:` Cambio incompatible (incrementa MAJOR version)
- `docs:` Cambios en documentación (no afecta versión)
- `refactor:` Refactorización sin cambio funcional
- `test:` Añadir o modificar tests

Branching Strategy (GitHub Flow)

- **main:** Rama principal, siempre deployable. Protegida con branch protection rules.
- **feature/*:** Ramas de desarrollo. Naming: feature/fase-1-ingestion, feature/fix-epss-parser.
- **Pull Requests:** Obligatorios para merge a main. Requieren: CI green, 1 approval, no conflicts.
- **Releases:** Tags semánticos (v1.0.0) generados automáticamente con semantic-release.

8.10 Documentación

- **Docstrings:** Google Style en todas las funciones públicas. Validado con pydocstyle.
- **API Docs:** OpenAPI 3.1 auto-generada por FastAPI. Disponible en /docs (Swagger UI) y /redoc.
- **Architecture Decision Records:** ADRs en docs/adr/ documentando decisiones técnicas importantes.
- **README:** Quick start, requisitos, configuración, contribución. Badges de CI, coverage, versión.
- **Diagramas:** Mermaid embebido en Markdown para arquitectura y flujos. Versionado junto al código.

9. Stack Tecnológico Completo

9.1 Dependencias Python (pyproject.toml)

```
langgraph>=0.2.0langchain>=0.2.0anthropic>=0.40.0chromadb>=0.5.0pydantic>=2.0.0scikit-
learn>=1.4.0xgboost>=2.0.0shap>=0.45.0lime>=0.2.0presidio-analyzer>=2.2.0presidio-
anonymizer>=2.2.0streamlit>=1.40.0fpdf2>=2.7.0requests>=2.31.0aiohttp>=3.9.0tenacity>=8.2.0p
ython-dotenv>=1.0.0sqlite-utils>=3.35.0
```

9.2 APIs Externas Requeridas

API	Propósito	Autenticación
Anthropic Claude	LLM para razonamiento	API Key (ANTHROPIC_API_KEY)
NVD API 2.0	Datos de CVEs	API Key opcional (NVD_API_KEY)
GitHub GraphQL	Security Advisories	Personal Access Token (GITHUB_TOKEN)
FIRST EPSS	Scores de explotabilidad	Sin autenticación requerida
Tavily Search	Búsqueda OSINT	API Key (TAVILY_API_KEY)
Jira REST API	Creación de tickets	API Token + Email (JIRA_*)

9.3 Consideraciones de Hardware (MacBook Air M3 16GB)

- **RAM:** ChromaDB configurado para máximo 4GB de cache en memoria. Resto disponible para Python runtime y modelo ML.
- **CPU:** Apple M3 (8 cores) suficiente para inferencia XGBoost local. No se requiere GPU.
- **Storage:** SQLite para checkpointing y ChromaDB persistence. Estimado ~2GB para corpus completo.
- **Network:** Conexión estable requerida para APIs externas. Offline mode no soportado.

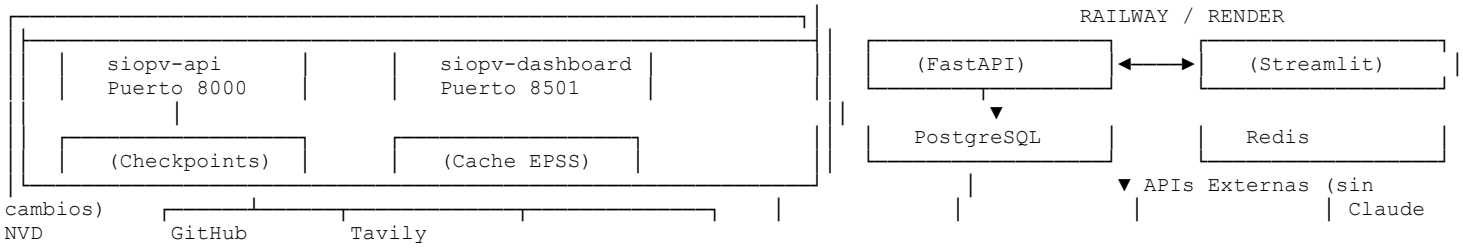
9.4 Despliegue en Cloud (Alternativa a Local)

Gracias al diseño 12-Factor App, SIOPV puede desplegarse en cualquier plataforma cloud sin modificar el código. Solo cambian las variables de configuración:

Plataformas Recomendadas

Plataforma	Ventajas	Caso de Uso Ideal
Railway	Deploy desde GitHub, PostgreSQL incluido, fácil escalado, buen free tier	Recomendado para demo del TFM. Deploy en 5 minutos.
Render	Free tier generoso, auto-deploy, managed PostgreSQL, SSL automático	Alternativa a Railway con tier gratuito más amplio.
Fly.io	Edge computing, persistent volumes, containers nativos	Si se requiere baja latencia geográfica.
Google Cloud Run	Serverless, escala a cero, integración GCP, free tier amplio	Producción enterprise con necesidades de escala.
Modal	Diseñado para ML/AI, serverless GPU/CPU, cron jobs nativos	Si se añaden modelos ML locales o fine-tuning.

Arquitectura Cloud Recomendada (Railway/Render)



Mapeo de Componentes Local → Cloud

Componente Local	Componente Cloud	Variable de Configuración
SQLite (fichero)	PostgreSQL (managed)	DATABASE_URL=postgresql://user:pass@host:5432/db
ChromaDB (persistente)	ChromaDB (HTTP client)	CHROMA_HOST=chroma.railway.internal
.env (fichero)	Environment Variables	Configuradas en dashboard de Railway/Render
localhost:8000	URL pública + SSL	Automático por la plataforma

El principio fundamental es que el código permanece idéntico; toda la diferencia entre entornos se gestiona mediante variables de configuración cargadas por Pydantic Settings.

10. Conclusiones y Siguientes Pasos

La propuesta técnica SIOPV v2.0 representa una evolución significativa respecto al diseño original, incorporando:

23. **Arquitectura de producción:** Migración de CrewAI a LangGraph para control determinista y trazabilidad completa.
24. **Pipeline híbrido robusto:** Combinación de ML clásico para scoring reproducible con LLMs para razonamiento semántico.
25. **Explicabilidad end-to-end:** LIME/SHAP para el modelo ML + Chain-of-Thought para decisiones del LLM.
26. **Seguridad por diseño:** Capas de autorización (ReBAC) y privacidad (DLP) integradas desde el inicio.
27. **Resiliencia operativa:** Circuit breakers, fallbacks y degradación graceful para entornos de producción.
28. **KPIs medibles:** Métricas cuantificables para demostrar el valor del sistema en la defensa del TFM.

10.1 Siguientes Pasos Inmediatos

29. Configurar entorno de desarrollo con todas las dependencias listadas.
30. Obtener API keys necesarias (Anthropic, NVD, GitHub, Tavily).
31. Descargar dataset CISA KEV e iniciar construcción del dataset de entrenamiento.
32. Implementar Fase 1 (Ingesta) como proof-of-concept inicial.
33. Establecer repositorio Git con estructura de proyecto y CI básico.

— Fin del Documento —